

Universidade Técnica Do Atlântico

Instituto de Engenharias e Ciências do Mar

Sidnei Cruz
scruz@uta.cv

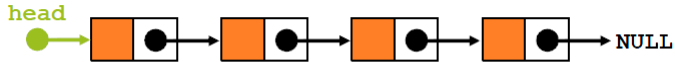


UNIVERSIDADE
TÉCNICA DO
ATLÂNTICO

CAMPUS
DO MAR

Lista Simplesmente Ligada

- Conjunto de nós



- Cada nó contém
 - Informação útil
 - Ponteiro para o próximo nó

```
struct node {  
    Item item;  
    struct node *next;  
} *link;
```



Lista Simplesmente Ligada

- representação encadeada
- Vantagens
 - Tamanho ilimitado (limite na memória disponível na máquina)
 - Alocação de memória apenas para os elementos que queremos representar
 - Inserção e remoção simples
- Desvantagens
 - Mais difícil o acesso ao n -ésimo elemento



UNIVERSIDADE
TÉCNICA DO
ATLÂNTICO

CAMPUS
DO MAR

Lista de String

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct node{
    char *text;
    struct node *next;
} *link;
```



UNIVERSIDADE
TÉCNICA DO
ATLÂNTICO



Lista Ligada

- Criação da lista vazia

```
void criar(link *head){  
  
    *head = NULL;  
}
```



UNIVERSIDADE
TÉCNICA DO
ATLÂNTICO

CAMPUS
DO MAR

Lista Ligada

- Criação da lista vazia

```
void criar(link *head){  
  
    *head = NULL;  
}
```

- Verificar se a lista está vazia

```
int vazia(link head){  
  
    return (head == NULL);  
}
```



UNIVERSIDADE
TÉCNICA DO
ATLÂNTICO

CAMPUS
DO MAR

Novo Elemento

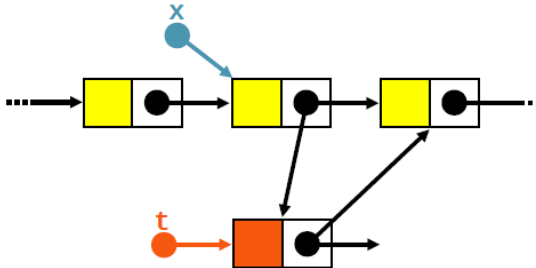
```
link NEW(char* text){  
  
    link x = (link) malloc(sizeof(struct node));  
    x->text = (char*) malloc(sizeof(char)*(strlen(text)+1));  
  
    strcpy(x->text, text);  
    x->next = NULL;  
  
    return x;  
}
```



Lista Simplesmente Ligada

- Inserção na Lista

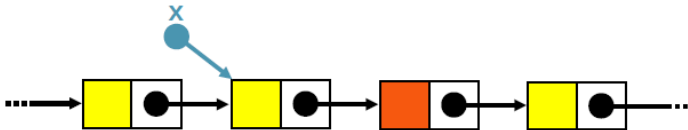
```
t->next = x->next;  
x->next = t;
```



Lista Simplesmente Ligada

- Inserção na Lista

```
t->next = x->next;  
x->next = t;
```



Inserção no Início

```
link insertBegin(link head, char* text){  
  
    link x = NEW(text);  
    x->next = head;  
  
    return x;  
}
```



UNIVERSIDADE
TÉCNICA DO
ATLÂNTICO

CAMPUS
DO MAR

Inserção no Início - Sem Valor de Retorno

```
link insertBegin(link head, char* text){  
  
    link x = NEW(text);  
    x->next = head;  
  
    return x;  
}
```



UNIVERSIDADE
TÉCNICA DO
ATLÂNTICO



Inserção no Início - Sem Valor de Retorno

```
link insertBegin(link head, char* text){  
  
    link x = NEW(text);  
    x->next = head;  
  
    return x;  
}
```

```
void insertBegin(link *headptr, char* text){  
  
    link x = NEW(text);  
    x->next = *headptr;  
  
    *headptr = x;  
}
```



Inserção no Fim

```
link insertEnd(link head, char* text){  
    link x;  
  
    if(head == NULL)  
        return NEW(text);  
  
    for(x = head; x->next != NULL; x = x->next)  
        ;  
    x->next = NEW(text);  
  
    return head;  
}
```



UNIVERSIDADE
TÉCNICA DO
ATLÂNTICO

CAMPUS
DO MAR

Procura na Lista

```
link procura(link head, char* text){  
    link t;  
  
    for(t = head; t != NULL; t = t->next)  
        if(strcmp(t->text, text) == 0)  
            return t;  
  
    return NULL;  
}
```

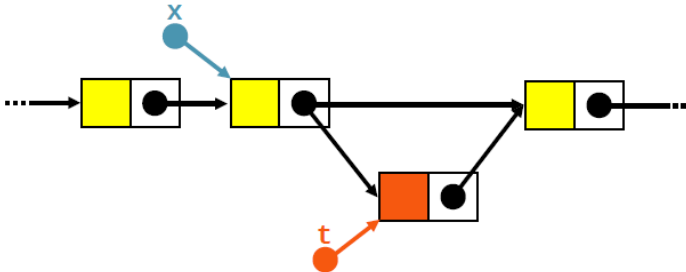


UNIVERSIDADE
TÉCNICA DO
ATLÂNTICO

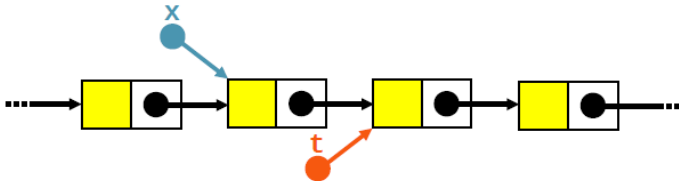
CAMPUS
DO MAR

Remoção da Lista

```
t = x->next;  
x->next = t->next;  
free(t);
```



Remoção da Lista com Procura

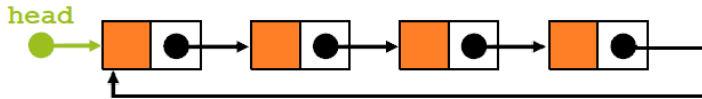


Remoção da Lista com Procura

```
link delete(link head, char* text){  
  
    link t, x;  
  
    for(t = head, x = NULL; t != NULL; x = t, t = t->next){  
        if(strcmp(t->text, text) == 0){  
            if(t == head)  
                head = t->next;  
            else  
                x->next = t->next;  
            free(t);  
        }  
    }  
  
    return head;  
}
```

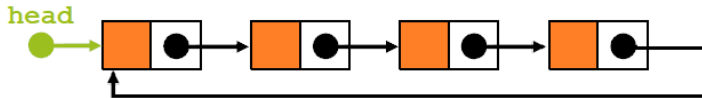
Lista Ligada

- Lista Circular

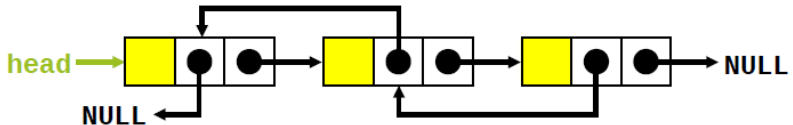


Lista Ligada

- Lista Circular

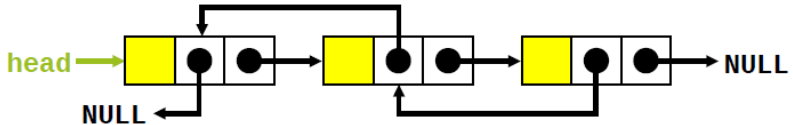


- Lista Duplamente Ligada



Lista Duplamente Ligada

- Conjunto de nós



- Cada nó contém
 - Informação útil
 - Ponteiro para o próximo nó e para o nó anterior

```
struct node {  
    Item item;  
    struct node *prev, *next;  
};
```